

1 Documentação técnica completa da Samsung Xclipse 940

Projeto: XO940

Hardware: Samsung Xclipse 940 no SM-S721B / plataforma s5e9945

Objeto: documentação técnica para estudo de plataforma, kernel SGPU/DRM, memória, command submission, Android e futuras implementações independentes

Status: evidência derivada de runtime, Device Tree, fonte Samsung inventariada e experimentos de cliente DRM; não constitui especificação oficial nem prova de driver Vulkan independente

1.1 Resumo técnico

A Xclipse 940 observada utiliza um caminho gráfico Samsung denominado SGPU, integrado ao kernel Linux/Android com estruturas e nomenclatura derivadas de AMDGPU. O dispositivo aparece no Device Tree como `sgpu@22200000`, referencia o domínio de energia `pd_g3dcore`, utiliza o render node `/dev/dri/renderD128` e expõe recursos de GFX e COMPUTE pela camada DRM.

A documentação atual confirma quatro camadas distintas. A primeira é a infraestrutura de plataforma, incluindo Device Tree, energia, clocks, IRQs, memória reservada, IOMMU e DMA. A segunda é o caminho de kernel/DRM, incluindo GEM/BO, VM, contextos, scheduler, IBs, rings e sincronização. A terceira é a pilha Android vendor, incluindo `libdrm_sgpu.so`, `vulkan.samsung.so`, OpenCL e namespaces. A quarta é a fronteira de um cliente independente, que já consegue abrir o render node, criar recursos DRM, mapear VA, criar `BO_LIST`, criar contexto e obter aceitação de uma entrada específica de CS, mas ainda não possui execução GPU validada por readback.

1.2 Política de evidência

A documentação diferencia observação direta, confirmação por fonte, resultado parcial, indício estático, hipótese e resultado negativo específico. A presença de uma biblioteca, símbolo, inicializador, pipeline ou retorno `VK_SUCCESS` antes da execução não comprova trabalho executado na GPU.

Uma afirmação de execução GPU exige a identificação do dispositivo-alvo, um recurso visível para a GPU, submissão para a fila correta, sincronização de conclusão e validação independente do resultado por readback ou apresentação. Os logs crus, fontes de teste, binários e dumps permanecem fora da distribuição pública.

1.3 1. Identidade do dispositivo

Campo	Valor observado	Classificação
Modelo	SM-S721B / r12s	Confirmado em runtime
Plataforma	erd9945 / s5e9945	Confirmado em runtime e fonte
GPU	Samsung Xclipse 940 / família MGFX 147	Confirmado na captura
Device ID	0x000073a0	Confirmado na captura
Chip revision	0x02600200	Confirmado na captura

Campo	Valor observado	Classificação
GFX IP	10.0 / ring mask 0xf	Confirmado na captura
COMPUTE IP	10.0 / ring mask 0x7	Identidade observada; execução independente não demonstrada
Firmware SGPU	2.23.0	Confirmado na captura
RTL	0x0004ea15	Confirmado na captura
Kernel	6.1.157-android14-11	Confirmado no ambiente analisado

1.4 2. Device Tree e plataforma

O nó principal é `/sys/firmware/devicetree/base/sgpu@22200000`, com `compatible = samsung-sgpu,samsung-sgpu`. Os filhos observados incluem `gpu_pm`, `gpu_doorbell`, `gpu_debug`, `gpu_smntarg` e `gpu_sysreg`. As regiões nomeadas incluem `gpu`, `doorbell`, `debug`, `pwrctl`, `sysreg` e `htu`.

As interrupções observadas são SGPU e GPU-AFM. O nó referencia `pd_g3dcore` por `phandle`. A plataforma também registra propriedades associadas a DVFS, IFPO, thermal G3D, memória reservada `sgpu_rmem`, `gpu_buffer_dma_heap`, grupos SysMMU/IOMMU e BTS G3D.

A Device Tree descreve recursos e dependências de plataforma. Ela não define sozinha a ISA, o formato completo de packets, a ABI de userspace ou a semântica de cada registrador.

1.5 3. Kernel SGPU, DRM e UAPI

Os caminhos de fonte mais relevantes são:

```
kernel/include/uapi/drm/sgpu_drm.h
kernel/drivers/gpu/drm/samsung/gpu/sgpu/
  amdgpu_drv.c
  amdgpu_cs.c
  amdgpu_vm.c
  amdgpu_gem.c
  amdgpu_object.c
  amdgpu_sched.c
  amdgpu_ring*.c
  amdgpu_ufcode.c
  amdgpu_device.c
  exynos_gpu_interface.c
kernel/arch/arm64/boot/dts/exynos/s5e9945-sgpu_common.dtsi
kernel/arch/arm64/boot/dts/exynos/s5e9945-sgpu_evt0.dtsi
kernel/drivers/iommu/samsung/
kernel/drivers/dma-buf/heaps/samsung/
```

A nomenclatura AMDGPU-derived indica reaproveitamento conceitual e estrutural. Ela não garante compatibilidade binária ou semântica com AMDGPU upstream, RADV ou qualquer backend Mesa existente.

1.6 4. Memória, GEM, BO, VA e IOMMU

O caminho observado registra criação e movimentação de BO, atualização de PTE/PDE, VM update e VM flush. Um cliente próprio criou um BO GTT de 64 KiB com alinhamento de 4096 bytes, realizou CPU mmap e fechou o objeto.

Os novos experimentos confirmaram adicionalmente:

Operação	Resultado
Criação de GEM/BO	Aceita pelo DRM
CPU mmap	Aceita pelo DRM
VA map/unmap	Aceito para VA 0x4000000000
VA map executável	Aceito com VA_MAP_EXECUTABLE=SUCCESS
BO_LIST create/destroy	Aceito pelo DRM
Context create/free	Aceito pelo DRM

Esses resultados confirmam o contrato básico de preparação de recursos. Eles não comprovam que a GPU acessou o BO, que o cache foi sincronizado ou que um page fault foi recuperado.

1.7 5. Command submission, IB e scheduler

Os traces de produção P4.5, P4.6 e P4.12 registram amdgpu_cs_ioctl, amdgpu_sched_run_job e amdgpu_ib_schedule no timeline/ring gfx_0.0.0, com sched_job, context, seqno e num_ibs=3. Essa é evidência real de atividade de submissão GFX no caminho vendor.

A nova rodada de cliente próprio registrou, em A1.5_REAL_CS_20260907_161914, uma aceitação específica do ioctl de CS:

```
DEVICE=/dev/dri/renderD128
BO handle=0x1 size=0x10000
IB va=0x4000000000 bytes=4
VA_MAP ioctl_ret=0 flags=0xe
BO_LIST list_handle=0x1
CTX ctx_id=1 priority=0
chunk_id=0x1 length_dw=8
ib_bytes=4 ip_type=0 ring=0
num_chunks=1
CS ioctl_ret=0 errno=0
CS_IOCTL=ACCEPTED
```

A interpretação correta é que o KMD aceitou essa estrutura de entrada de userspace. O resultado não comprova a execução do packet de 4 bytes, a interpretação pelo command processor, o avanço de ring, a sinalização de fence ou a escrita no BO.

Outras tentativas foram rejeitadas com EINVAL ou EFAULT/Bad address. Essas rejeições documentam a validação de estruturas e ponteiros pelo KMD, mas não provam que seja bloqueado qualquer CS próprio seja bloqueado.

1.8 6. Sincronização e conclusão

A infraestrutura possui referências a fences, syncobjs, dependências SGPU, scheduler e timelines. Os traces de produção indicam ciclo de vida de jobs e eventos relacionados a fence. A captura do cliente próprio ainda não correlaciona o CS aceito com uma out-fence ou syncobj sinalizado.

A cadeia de conclusão ainda aberta é:

```
CS aceito → job no scheduler → IB/ring → execução GPU
→ fence/out-fence/syncobj → retire → readback
```

O agendamento de um job ou a aceitação do ioctl não deve ser descrito como fence concluída.

1.9 7. Android, loader e pilha vendor

O SurfaceFlinger observado mapeia /system/lib64/libvulkan.so e /vendor/lib64/hw/vulkan.samsung.so, além de manter FDs para /dev/dri/renderD128. O serviço com.samsung.android.photoremasterservice:photoremasterservice carregou vulkan.samsung.so, libdrm_sgpu.so, libOpenCL.so e libSGPUOpenCL.so.

O Termux opera com namespace de linker/mount diferente e expôs llvmpipe. Root e SELinux permissivo temporário não transformaram o processo em membro do namespace vendor. Portanto, carregar o ICD a partir de um processo externo exige uma ponte Android suportada ou uma estratégia de integração que respeite ABI, permissões e namespaces.

1.10 8. OpenCL, Vulkan e compute

A pilha vendor contém APIs e exports para contexto, filas, buffers, programas, kernels, clEnqueueNDRangeKernel, clEnqueueReadBuffer, clBuildProgram e clCreateProgramWithIL. Isso confirma uma superfície compute instalada e um consumidor vendor real.

Ainda não foi demonstrado um dispatch compute próprio com entrada conhecida, submissão, fence e readback independente. O probe Vulkan analisado criou metadados de pipeline, mas não fechou queue submission, dispatch, apresentação ou readback no dispositivo Samsung.

1.11 9. Compiler, SPIR-V e ISA

O inventário estático do ICD contém strings e referências associadas a SPIR-V, shader compiler, opcode translation, emitter, encoder, VOPC/VOP1/VOP2/VOP3/VOP3P e DPP8/DPP16. Isso constitui evidência estática de infraestrutura de compilação e encoding.

Não existe ainda uma correlação runtime entre um shader controlado, o binary produzido, uma instrução nativa da Xclipse e um resultado validado. Os nomes de componentes semelhantes a AMD não devem ser tratados como prova de ISA AMDGPU compatível.

1.12 10. Estado técnico consolidado

Área	Estado atual
Device Tree e plataforma	Confirmado em grande parte
DRM render node e UAPI	Confirmado para o caminho observado
GEM/BO/CPU mmap	Confirmado em cliente próprio
VA map/unmap	Confirmado em cliente próprio
BO_LIST	Confirmado em cliente próprio
Context create/free	Confirmado em cliente próprio
CS ioctl próprio	Aceitação confirmada para uma entrada específica
Execução GPU própria	Não demonstrada
Fence/readback próprios	Não demonstrados
Compute independente	Não demonstrado
ISA nativa	Indícios estáticos
Backend NIR/ACO	Não implementado
Integração RADV	Não implementada
Graphics pipeline próprio	Não demonstrado
WSI/presentação	Não demonstrado

1.13 11. Limites de publicação

A documentação pública contém conclusões sanitizadas, tabelas, mapas e descrições técnicas. Não contém testes baixáveis, fontes C dos clientes, executáveis, bibliotecas vendor, firmware bruto, traces crus, dumps de memória ou dmesg completo.

1.14 Referências

[1](#) [2](#) [3](#) [4](#)

Este documento é uma síntese técnica para estudo. Ele não declara a existência de um driver independente funcional.